

MODULE 26 · EXAM STRATEGY - ALL DOMAINS

Practice exam 3

The last twenty questions, the trend across all sixty, and what to do next.

DOMAIN All four QUESTIONS 20 MODULE 26 of 27

Before you start

- ◆ **Review exams one and two first** if you have not already.
- ◆ **Taking this cold** measures the same gaps a third time.
- ◆ **Forty minutes**, and note the time you finish as well as the score.
- ◆ **Record the letter you chose** for each question, as before.
- ◆ **No question repeats** from exams one or two.

ON THE EXAM

The value of a third exam depends entirely on what happened after the second. Review first, then sit this.

What the three exams cover between them

- ◆ **Sixty questions** across the four domains in the published weighting.
- ◆ **Six multiple-response questions**, two in each set.
- ◆ **Every in-scope service group** appears somewhere across the three.
- ◆ **Several decisions appear twice**, with one clause changed so the answer moves.
- ◆ **Sixty questions is a small sample**. It finds patterns, not a predicted score.

ON THE EXAM

Question 12 here is deliberately the mirror of a partition question in exam one. One changed clause, different answer.

A company needs order events available in Amazon Redshift within seconds for an operational dashboard. Events are produced to Amazon Kinesis Data Streams. The team does not want to stage files in Amazon S3 or operate a consumer application. Volumes are moderate and the dashboard queries aggregate the most recent events.

What should the data engineer implement?

- ☐ A Amazon Data Firehose delivering to Amazon S3, with a scheduled COPY into Redshift
- ☐ B Redshift streaming ingestion, materialising the stream into a materialized view
- ☐ C An AWS Lambda consumer writing rows to Redshift with the Data API
- ☐ D AWS Glue streaming job writing to Redshift every minute

◆ PAUSE AND DECIDE

What should the data engineer implement?

- A

 Firehose to S3, then COPY
Stages files in S3, which the question rules out, and adds buffering plus a scheduled load to a seconds requirement.
- B

 Redshift streaming ingestion
Connects the warehouse directly to the stream, materialising records into a view with no S3 staging and no consumer to run.
- C

 A Lambda consumer using the Data API
A consumer application to build, deploy and operate, which the question explicitly excludes.
- D

 A Glue streaming job
Also a consumer to operate, and a minute of latency where seconds were asked for.

THE SIGNAL

No S3 staging and no consumer to operate. Those two constraints together leave one option.

An AWS Glue job created two years ago still runs with default settings. It processes about eight gigabytes nightly. Monitoring shows executor utilisation below fifteen percent throughout, and the job completes in twelve minutes.

A cost review has flagged it as one of the more expensive jobs in the account relative to the data it handles.

What should the data engineer do?

- ☐ A Switch the job to Glue Flex execution
- ☐ B Enable job bookmarks
- ☐ C Move the job to Amazon EMR Serverless
- ☐ D Reduce the number of workers from the default allocation to match observed utilisation

◆ PAUSE AND DECIDE

What should the data engineer do?

A Glue Flex

A discount on capacity that is eighty five percent idle. It reduces the rate rather than the over-provisioning.

C EMR Serverless

Introduces a different service to solve a configuration problem inside the one already running the job.

B Job bookmarks

Prevents reprocessing. Nothing here suggests the job is re-reading history; it processes eight gigabytes in twelve minutes.

D Right-size the worker count

Fifteen percent utilisation on default settings means the job was never sized. Matching workers to the work is the direct fix.

THE SIGNAL

Two years old, default settings, fifteen percent utilisation. The defaults are the answer.

An AWS Lambda function is triggered per file arriving in Amazon S3 and writes results to an Amazon RDS for PostgreSQL database. During bulk uploads, thousands of files arrive at once and Lambda scales to hundreds of concurrent executions.

The database exhausts its connection limit and the application that also uses it begins failing.

What should the data engineer implement?

- ☐ A Increase the RDS instance size to raise the connection limit
- ☐ B Enable provisioned concurrency on the Lambda function
- ☐ C Set reserved concurrency on the Lambda function to cap simultaneous executions
- ☐ D Increase the Lambda function timeout

◆ PAUSE AND DECIDE

What should the data engineer implement?

A A larger RDS instance

Buys headroom and moves the ceiling. A larger bulk upload exhausts the new limit and the application fails again.

C Reserved concurrency

Caps how many functions run at once, protecting the shared database. The work takes longer and the application stays up.

B Provisioned concurrency

Keeps environments warm to remove cold starts. It does not limit concurrency; it can increase readiness to scale.

D A longer timeout

Nothing is timing out. Functions are failing to obtain a connection, which happens immediately.

THE SIGNAL

Parallelism upstream became a bottleneck downstream. Reserved concurrency is a throttle you apply to yourself.

A company must process around four million individual small JSON objects in Amazon S3, applying a transformation to each and writing the result back. The work is embarrassingly parallel and each object takes under a second.

The team wants to run this without building custom fan-out logic, and needs per-object success and failure tracking.

What should the data engineer use?

- ☐ A An AWS Glue Spark job listing the prefix and processing every object
- ☐ B A single Lambda function iterating over the objects in a loop
- ☐ C A Step Functions inline Map state over a list of four million keys
- ☐ D A Step Functions Distributed Map state invoking a Lambda function per object

◆ PAUSE AND DECIDE

What should the data engineer use?

A A Glue Spark job

Workable, and Spark on four million tiny objects spends most of its time listing and opening files rather than transforming.

C An inline Map state

Inline Map runs within the parent execution with limited concurrency and a payload ceiling. Four million items exceeds it.

B One Lambda looping

Four million objects at under a second each is well past the fifteen minute limit, and it is serial.

D Distributed Map with a Lambda per object

Reads the object list from S3, fans out at high concurrency, and reports per-item success and failure. No custom fan-out to build.

THE SIGNAL

Millions of independent objects, no custom fan-out, per-item tracking. That combination names Distributed Map.

Sixty suppliers send daily inventory files to a retailer. Each supplier uses SFTP with its own credentials and cannot change how it transmits files. The retailer wants the files to land in Amazon S3 and to stop operating the current SFTP servers on Amazon EC2. File sizes are modest and arrive throughout the day.

What should the data engineer implement?

- ☐ A AWS Transfer Family with an SFTP endpoint backed by Amazon S3
- ☐ B AWS DataSync agents at each supplier site
- ☐ C Amazon AppFlow with an SFTP connector
- ☐ D AWS Snowball Edge devices shipped to each supplier

◆ PAUSE AND DECIDE

What should the data engineer implement?

- A

AWS Transfer Family
A managed SFTP endpoint writing straight into S3. Suppliers keep their protocol and credentials; the EC2 servers go away.
- B

DataSync agents at each site
Requires software deployed on sixty third-party networks, and changes how every supplier transmits.
- C

AppFlow with SFTP
AppFlow connects to SaaS applications. SFTP file transfer is not what it does.
- D

Snowball Edge
Physical devices for very large offline transfers. These are modest daily files, and Snowball is closed to new customers.

THE SIGNAL

"Cannot change how it transmits" plus SFTP names Transfer Family. The protocol is the requirement.

A Glue job writes to a curated Amazon S3 table used by Athena. Output consists of roughly forty thousand files per day, most under two hundred kilobytes, because the job writes one file per input partition. Athena queries have become slow, spending most of their time before any data is read.

The data is Parquet and correctly partitioned by date.

Select TWO actions that will improve query performance.

- ☐ A Coalesce or repartition the output so the job writes fewer, larger files
- ☐ B Run a scheduled compaction job over existing partitions
- ☐ C Convert the output from Parquet to CSV
- ☐ D Add more workers to the Glue job

◆ PAUSE AND DECIDE

Select TWO actions that will improve query performance.

- A

 Coalesce the output
Fixes the cause, so tomorrow's output is a manageable number of files rather than forty thousand.
- B

 Compact existing partitions
Fixes the accumulated problem. Without it, every partition already written stays slow to query.
- C

 Convert to CSV
Abandons columnar storage. It would increase bytes scanned substantially.
- D

 More workers
More workers typically write more files, not fewer. It makes the small file problem worse.

THE SIGNAL

Stop creating the problem and clean up what exists. One answer for each half, which is the usual multi-response shape.

A data pipeline reads a two megabyte CSV file each hour, renames four columns, casts two dates, and writes Parquet. It currently runs as an AWS Glue Spark job with the default allocation and takes about four minutes, most of which is startup. The team wants the lowest cost for this specific step.

What should the data engineer use?

- ☐ A An AWS Lambda function triggered by the file arrival
- ☐ B Keep the Glue Spark job and reduce it to the minimum two DPUs
- ☐ C A Glue Python shell job
- ☐ D An Amazon EMR Serverless application

◆ PAUSE AND DECIDE

What should the data engineer use?

- A

A Lambda function
Megabytes, three trivial transformations, seconds of work. No startup overhead and no billing minimum beyond the milliseconds used.
- B

Two DPU's on Spark
A real improvement, and still a Spark cluster starting up for two megabytes, with a one minute billing minimum each hour.
- C

A Glue Python shell job
Genuinely close, and cheaper than Spark. It still carries Glue startup time and a one minute minimum, twenty four times a day.
- D

EMR Serverless
A Spark runtime for a two megabyte file. Heavier than the option it would replace.

THE SIGNAL

Two megabytes and three trivial transformations. Nothing here needs a distributed engine at all.

A company is launching a new product and will store its telemetry in Amazon S3. Nobody can predict how often the data will be read, and access patterns are expected to change as the product matures. Objects average four megabytes.

The team wants to minimise storage cost without risking retrieval charges from a wrong guess.

What should the data engineer configure?

- ☐ A A lifecycle rule transitioning to Standard-IA after thirty days
- ☐ B S3 Intelligent-Tiering
- ☐ C A lifecycle rule transitioning to Glacier Instant Retrieval after seven days
- ☐ D S3 One Zone-Infrequent Access from the outset

◆ PAUSE AND DECIDE

What should the data engineer configure?

- A

 Lifecycle to Standard-IA
A lifecycle rule encodes a guess about access. The question says nobody can predict it, and a wrong guess means retrieval charges.
- C

 Glacier Instant Retrieval after seven days
A more aggressive guess, with a ninety day minimum. Early access would be charged and early deletion penalised.
- B

 S3 Intelligent-Tiering
Moves objects between tiers based on observed access, with no retrieval fees and no minimum duration. Built for unknown patterns.
- D

 One Zone-IA from the outset
Assumes infrequent access from day one, and stores the only copy in a single Availability Zone.

THE SIGNAL

"Nobody can predict" and "expected to change" is the Intelligent-Tiering signal, almost word for word.

A nightly pipeline loads order records into an Amazon Redshift table that has a declared primary key on order ID. After a failure and rerun last week, the table now contains around twelve thousand duplicated orders and reports are overstated.

The engineer is surprised, because the primary key should have rejected them.

What is the cause, and what should be implemented?

- ☐ A The rerun used a different distribution key, creating duplicate slices
- ☐ B The primary key was declared after the table was created, so it applies only to new rows
- ☐ C Redshift constraints are informational and not enforced. Make the load idempotent with a staging table and a MERGE statement
- ☐ D Enable automatic table optimization so Redshift enforces the key

◆ PAUSE AND DECIDE

What is the cause, and what should be implemented?

- A

 A different distribution key
Distribution decides which slice holds a row. It does not duplicate rows.
- B

 Declared after creation
Invented. Primary keys in Redshift are never enforced, whenever they are declared.
- C

 Constraints are informational; use staging and MERGE
Redshift does not enforce primary keys. Idempotency has to come from the load, and COPY to staging then MERGE is the standard pattern.
- D

 Automatic table optimization
Manages sort and distribution keys based on query patterns. It does not enforce constraints.

THE SIGNAL

A declared primary key that did not prevent duplicates is not a bug. Redshift constraints are hints to the planner.

A trading platform needs a session store holding around two hundred gigabytes. Reads and writes must complete in microseconds, and the data must survive an Availability Zone failure without loss because it is the authoritative record for in-flight sessions.

The team currently uses an in-memory cache backed by a relational database.

What should the data engineer recommend?

- ☐ A Amazon DynamoDB with DynamoDB Accelerator
- ☐ B Amazon S3 Express One Zone
- ☐ C Amazon Aurora PostgreSQL with a large buffer cache
- ☐ D Amazon MemoryDB

◆ PAUSE AND DECIDE

What should the data engineer recommend?

- A

DynamoDB with DAX
DAX is a cache in front of DynamoDB, so it is not the authoritative record, and DynamoDB itself is single-digit milliseconds.
- B

S3 Express One Zone
Single-digit milliseconds, and a single Availability Zone, which the durability requirement rules out.
- C

Aurora with a large cache
Milliseconds at best, and a cache hit is not a durability guarantee.
- D

Amazon MemoryDB
In-memory speed with durability across Availability Zones, so it can be the primary store rather than a cache.

THE SIGNAL

Microseconds and durable and authoritative. That combination is MemoryDB rather than any cache.

An analytics team must rename a column in a five year old dataset of Parquet files in Amazon S3, because the business term has changed. Historical queries must continue to return the same rows, and rewriting five years of files is not acceptable.

The table is queried by Amazon Athena and Amazon EMR.

What should the data engineer do?

- ☐ A Rename the column in the Glue Data Catalog and leave the files unchanged
- ☐ B Create a view that aliases the old column name to the new one
- ☐ C Convert the dataset to Apache Iceberg, then rename the column
- ☐ D Rewrite all five years of Parquet files with the new column name

◆ PAUSE AND DECIDE

What should the data engineer do?

- A

 Rename in the catalog only
Parquet matches columns by name, so the catalog name would no longer match the files and the column would read as null.
- B

 A view aliasing the name
Workable for readers who use the view, and the underlying table keeps the old name, so anything querying it directly is unaffected.
- C

 Convert to Iceberg, then rename
Iceberg tracks columns by a stable numeric ID, so a rename changes the name while every historical file still maps correctly.
- D

 Rewrite five years of files
Achieves the rename by doing exactly what the question rules out.

THE SIGNAL

Column IDs are why a rename is safe in Iceberg and unsafe in plain Parquet. That is the mechanism being tested.

A table in the AWS Glue Data Catalog has around one hundred and forty thousand partitions after four years of hourly partitioning. Athena queries take fifteen to twenty five seconds before scanning begins, regardless of how narrow the filter is. Once scanning starts, they complete quickly.

Analysts filter by date range and occasionally by hour.

Which action addresses the delay while preserving the hourly granularity analysts occasionally use?

- ☐ A Repartition the data by day and discard the hour partition
- ☐ B Add a Glue partition index on the partition keys
- ☐ C Increase the Athena workgroup's query timeout
- ☐ D Run MSCK REPAIR TABLE to refresh the partition metadata

◆ PAUSE AND DECIDE

Which action addresses the delay while preserving the hourly granularity analysts occasionally use?

A Repartition by day

The direct fix for over-partitioning, and it discards the hourly granularity the question explicitly says analysts use.

C A longer timeout

Lets a slow query finish rather than making it fast.

B A Glue partition index

Speeds up partition lookup against a very large partition count, without removing the hourly granularity the question protects.

D MSCK REPAIR TABLE

Adds missing partitions. Nothing is missing, and running it across 140,000 partitions is itself slow.

THE SIGNAL

Compare with exam one's partition question. There the hour partition was unused; here it is needed, and the answer moves.

An application writes a log line containing the text `RECORD_REJECTED` whenever it discards a malformed record. These lines go to CloudWatch Logs. Operations want to be notified when the rate of rejections exceeds a threshold.

They have tried and failed to configure a CloudWatch alarm directly against the log group.

What should the data engineer implement?

- ☐ A CloudWatch Logs metric filter matching the pattern, with an alarm on the resulting metric
- ☐ A CloudWatch alarm configured with the log group as its metric source
- ☐ A CloudWatch Logs subscription filter delivering matching events to Amazon S3
- ☐ An EventBridge rule matching the log content

◆ PAUSE AND DECIDE

What should the data engineer implement?

- A

 A metric filter, then an alarm on the metric
Alarms evaluate metrics, not log content. A metric filter is the bridge, counting matching events into a metric an alarm can watch.
- B

 An alarm on the log group
Not possible. This is what the team already tried and could not configure.
- C

 A subscription filter to S3
Streams matching events onward for storage or processing. It creates no metric and raises no alert.
- D

 An EventBridge rule on log content
EventBridge matches events services emit, not arbitrary text inside application log lines.

THE SIGNAL

The team's failed attempt is the clue. Alarms cannot watch logs directly, which is exactly why metric filters exist.

An executive dashboard built on Amazon Redshift is opened by around eight hundred people between eight and nine each morning. The underlying data is refreshed once overnight and does not change during the day. During that hour, other analytical workloads on the cluster slow noticeably.

Every viewer's dashboard runs the same set of queries.

What is the most appropriate change?

- ☐ A Import the dashboard dataset into SPICE and refresh it after the overnight load completes
- ☐ B Enable Redshift concurrency scaling so additional clusters absorb the morning burst
- ☐ C Resize the Redshift cluster to handle the peak
- ☐ D Point the dashboard at Amazon Athena over the same data in Amazon S3

◆ PAUSE AND DECIDE

What is the most appropriate change?

- A

 Import into SPICE, refresh after the load
Eight hundred identical queries against static data become zero. The dashboard reads a cached copy and the cluster is untouched.
- B

 Concurrency scaling
Absorbs the burst by adding clusters, and pays to run eight hundred identical queries that never needed running.
- C

 Resize the cluster
Permanent capacity for a one hour event, and the redundant queries still run every morning.
- D

 Point the dashboard at Athena
Moves the load rather than removing it. Eight hundred scans of the same data, now billed per terabyte.

THE SIGNAL

Identical queries against data that does not change is the caching signal. Removing work beats absorbing it.

A pipeline loads financial transactions into a warehouse used for regulatory reporting. Occasionally a small number of records arrive with a missing settlement currency. Partial figures are worse than no figures for this use case, because reports built on incomplete data have twice been filed.

The team wants to catch this and does not want to write custom validation code.

Select TWO actions the data engineer should take.

- ☐ A Add AWS Glue Data Quality rules inside the ETL job checking completeness of the currency column
- ☐ B Configure the job to fail when the ruleset does not pass, so nothing downstream is loaded
- ☐ C Configure the job to route failing records to a quarantine location and continue
- ☐ D Add a CloudWatch alarm on the Glue job's failed run count

◆ PAUSE AND DECIDE

Select TWO actions the data engineer should take.

- A

 Glue Data Quality rules in the job
A declarative completeness rule, evaluated as data flows through, with no custom validation code to maintain.
- B

 Fail the job on violation
Partial figures are worse than none for regulatory reporting, so stopping is correct and quarantining would publish incomplete data.
- C

 Quarantine and continue
Right when partial data has value. Here the question states the opposite, and incomplete reports have already been filed.
- D

 Alarm on failed runs
Useful once the job fails, and on its own it detects nothing, because today the job succeeds with bad data.

THE SIGNAL

"Partial figures are worse than no figures" is the clause choosing fail over quarantine. It is the only difference.

An analyst accidentally ran a SELECT star query against a forty terabyte unpartitioned table in Amazon Athena, producing a single query costing several hundred dollars. Management wants to prevent a recurrence.

The underlying data is scheduled to be converted to partitioned Parquet next quarter.

What should the data engineer configure now?

- ☐ A Remove the analyst's Athena permissions
- ☐ B An AWS Budget alert on the Athena service
- ☐ C A per-query data scan limit on the Athena workgroup
- ☐ D Increase the workgroup's query timeout so large queries are less likely to be retried

◆ PAUSE AND DECIDE

What should the data engineer configure now?

- A

 Remove the permissions
Prevents recurrence by preventing the analyst's job. It also does nothing about anybody else.
- B

 A Budget alert
Tells you afterwards that spend crossed a line. The damage from one query is already done.
- C

 A per-query data scan limit
Cancels any query that would read more than a set volume, so a single accidental query cannot run up a large bill.
- D

 A longer timeout
Allows expensive queries to run for longer. It is the opposite of a guardrail.

THE SIGNAL

Prevent or detect? A scan limit refuses the query. A budget alert reports the bill afterwards.

A company has fifteen AWS accounts and around sixty engineers. Each engineer has an IAM user with access keys in every account they work in. Two people left last quarter and the security team cannot confirm all their credentials were removed.

The company wants access to end when employment ends, and to stop issuing long-lived keys.

What should the company implement?

- ☐ A A scheduled Lambda function deleting IAM users inactive for sixty days
- ☐ B A ninety day access key rotation policy enforced across all accounts
- ☐ C AWS IAM Identity Center federated to the corporate directory, with permission sets granting roles per account
- ☐ D Consolidate the fifteen accounts into three

◆ PAUSE AND DECIDE

What should the company implement?

A Delete inactive users

Cleans up on a sixty day delay, during which a departed engineer's key remains valid.

C Identity Center federated to the directory

Identity lives in one place tied to employment, and access is temporary role credentials rather than keys.

B Ninety day key rotation

Shortens the life of a leaked key while keeping long-lived keys in existence, which the company said it wants to stop.

D Fewer accounts

Reduces the number of places to clean up, and does nothing about identity being tied to employment.

THE SIGNAL

"Access should end when employment ends" is a statement about where identity lives. Only federation satisfies it.

An insurer holds claims data in Amazon Redshift. Regional teams must see only their own region's claims. A fraud team must see all regions. A support team must see all regions but with the claimant's national identifier partially masked, while the fraud team sees it in full. There are currently eleven views maintaining these combinations, and keeping them aligned with schema changes has become a burden.

What should the data engineer implement?

- ☐ A Create one table per region with the identifier column omitted where required
- ☐ B Continue with views, generating them automatically when the schema changes
- ☐ C Move the data to Amazon S3 and apply Lake Formation data filters
- ☐ D Row-level security policies for the regional filtering, and dynamic data masking on the identifier column by role

◆ PAUSE AND DECIDE

What should the data engineer implement?

- A

 One table per region
Multiplies the schema, the pipelines and the drift, and still does not address the two masking audiences.
- B

 Generate the views automatically
Automates the maintenance of eleven objects rather than removing them. The duplication remains.
- C

 Move to S3 and use Lake Formation
Relocates warehouse data to the lake to reach capabilities Redshift already has natively.
- D

 Row-level security plus dynamic data masking
One table, one schema. Rows filter by the session's role and the identifier masks by role, with nothing to keep aligned.

THE SIGNAL

Two distinct requirements, two native Redshift features. The eleven views exist because neither was used.

A data lake of around fifty million Parquet objects is encrypted with SSE-KMS using a customer managed key. Compliance requires a customer managed key with an audit trail, so the encryption cannot be changed.

Finance has flagged that the AWS KMS bill scales with query volume rather than with the amount of data stored.

What should the data engineer do?

- ☐ A Move the data to S3 Standard-Infrequent Access
- ☐ B Switch the bucket to SSE-S3 encryption
- ☐ C Request a KMS request quota increase
- ☐ D Enable S3 Bucket Keys on the bucket

◆ PAUSE AND DECIDE

What should the data engineer do?

A Standard-IA

Changes storage cost and adds a retrieval charge. The KMS request volume is unaffected.

C A quota increase

Raises the request ceiling. It does nothing about being billed for every one of those requests.

B Switch to SSE-S3

Removes the customer managed key and its audit trail, which compliance requires. It solves cost by breaking the requirement.

D Enable S3 Bucket Keys

A bucket-level key derives per-object keys locally, cutting KMS requests by up to ninety nine percent with the same key and encryption.

THE SIGNAL

A KMS bill scaling with query volume rather than data volume is the per-object decryption signature.

A large organisation has around eight hundred datasets across six business units. Access requests are handled by a central platform team, who receive around forty a week and take several days each. The team approving requests often does not know whether the requester should have the data.

The organisation wants requests decided by whoever owns the data, with each decision recorded for audit.

What should the data engineer implement?

- ☐ A Grant each business unit database-level Lake Formation permissions on its own databases
- ☐ B Publish datasets as assets in Amazon SageMaker Catalog and use the subscription workflow, with data owners approving requests
- ☐ C Add more engineers to the central platform team
- ☐ D Give every analyst read access to all eight hundred datasets and audit usage with CloudTrail

◆ PAUSE AND DECIDE

What should the data engineer implement?

- A

 Database-level grants per business unit
Removes the queue by granting broadly. It cannot distinguish sensitivity within a business unit, and records no decision.
- B

 SageMaker Catalog subscription workflow
Consumers request, the data owner approves, and Lake Formation permissions are applied automatically with the decision recorded.
- C

 More engineers
Scales the bottleneck. The approvers still lack the context to decide, which was the stated problem.
- D

 Access to everything, audited afterwards
Abandons access control and substitutes detection. It is the opposite of what the requirement describes.

THE SIGNAL

Two requirements: the owner decides, and the decision is recorded. The subscription workflow does both.

Which module each question came from

QUESTIONS	REVISIT
1	Module 3, streaming ingestion
2, 4, 6, 7	Modules 2, 5 and 6, Glue, engines and orchestration
3	Modules 6 and 7, concurrency and Lambda
5	Module 4, migration and connectivity
8, 9, 10, 11, 12	Modules 8 to 12, storage, Redshift, stores, Iceberg, catalog
13, 14, 15, 16	Modules 13 to 16, querying, monitoring, quality, cost
17, 18, 19, 20	Modules 17 to 20, auth, authorisation, encryption, governance

ON THE EXAM

Compare all three exams. A row you have now missed twice is where your remaining preparation time belongs.

The distractor patterns used here

PATTERN	QUESTIONS WHERE IT APPEARED
Capacity for a non-capacity problem	3, 6, 14
Right answer, wrong question	2, 12, 16
Breaks a stated constraint	1, 5, 11, 19
Solves half the requirement	15, 20
Asserting something false	9, 13
The option you would build in production	7, 18

ON THE EXAM

Across all three exams, breaking a stated constraint was the most common way a wrong option was made attractive.

Counting your answers by letter

- ◆ **Correct answers are spread evenly** across A, B, C and D in all three sets.
- ◆ **If your choices cluster on one letter**, that is about you rather than the questions.
- ◆ **A bias towards A and B** usually means deciding before reading all four options.
- ◆ **A bias towards longer options** is a known habit worth checking for.
- ◆ **Read all four before choosing**, even when the first one looks right.

ON THE EXAM

Answer letters were deliberately rebalanced in all three exams so a positional habit gains you nothing here.

Reading your trend across sixty questions

- ◆ **Improving across the three** means reviewing is working. Keep doing exactly that.
- ◆ **Flat across the three** usually means you scored without reviewing in between.
- ◆ **Strong on facts, weak on scenarios** points at reading order rather than knowledge.
- ◆ **Strong on Domain 1, weak on Domain 4** is the most common shape, and the easiest to fix.
- ◆ **One category of mistake repeating** is worth more attention than any single domain.

ON THE EXAM

Sixty questions is enough to see a pattern. It is not enough to predict a scaled score, and nobody can honestly tell you otherwise.

What to do with the last week

- ◆ **Reread module twenty one**, particularly the words that decide.
- ◆ **Reread module twenty three**, the availability warnings and the renames.
- ◆ **Work through every question you got wrong** until you can explain all four options.
- ◆ **Do not learn new services.** Nothing absorbed in the final week survives to the exam.
- ◆ **Sleep, and check the logistics.** Fatigue costs more marks than one more service gains.

ON THE EXAM

The third point is the whole method. Being able to say why each wrong option is wrong is the skill the exam tests.

You have covered **every task statement** in the exam guide, and about a hundred worked scenarios.

What this course tried to do

- ◆ **Teach the decisions**, rather than the service descriptions.
- ◆ **Name the deciding sentence** in every worked scenario.
- ◆ **Verify every fact** against AWS documentation, with the date recorded.
- ◆ **Say plainly where the exam and reality diverge**, rather than teaching one.
- ◆ **Cover Domain 4 properly**, because it is where most candidates lose marks.

ON THE EXAM

Good luck. The reasoning transfers well beyond the exam, which is rather the point.